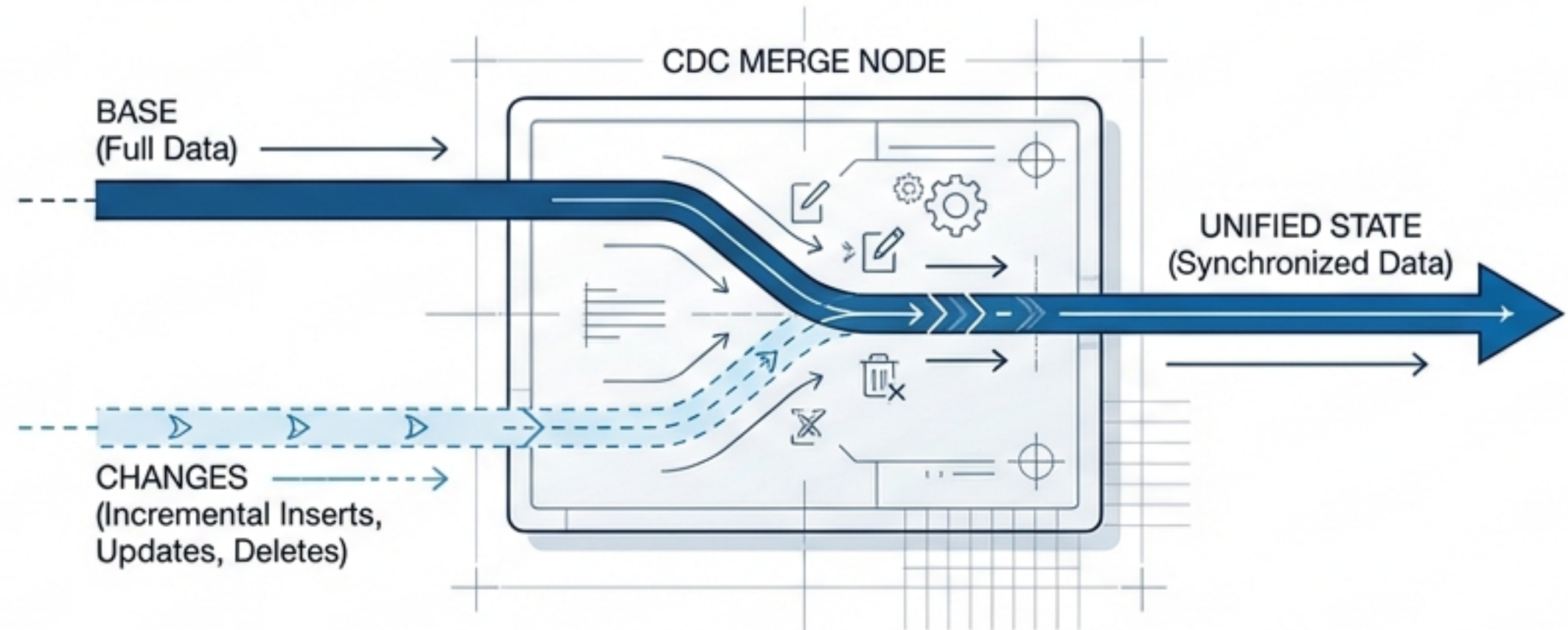


Building a Scalable Change Data Capture (CDC) Pipeline with PySpark

Efficiently synchronizing MySQL employee data by processing only incremental inserts, updates, and deletes.

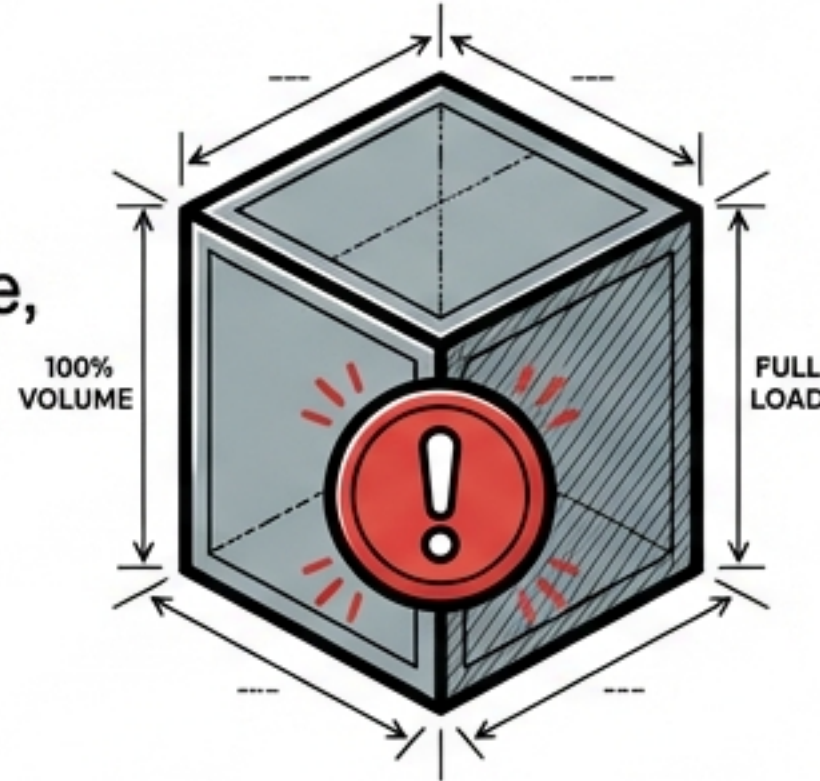


A technical walkthrough of a batch processing architecture that eliminates full table refreshes in favor of precision data merging.

The Challenge of Data Synchronization

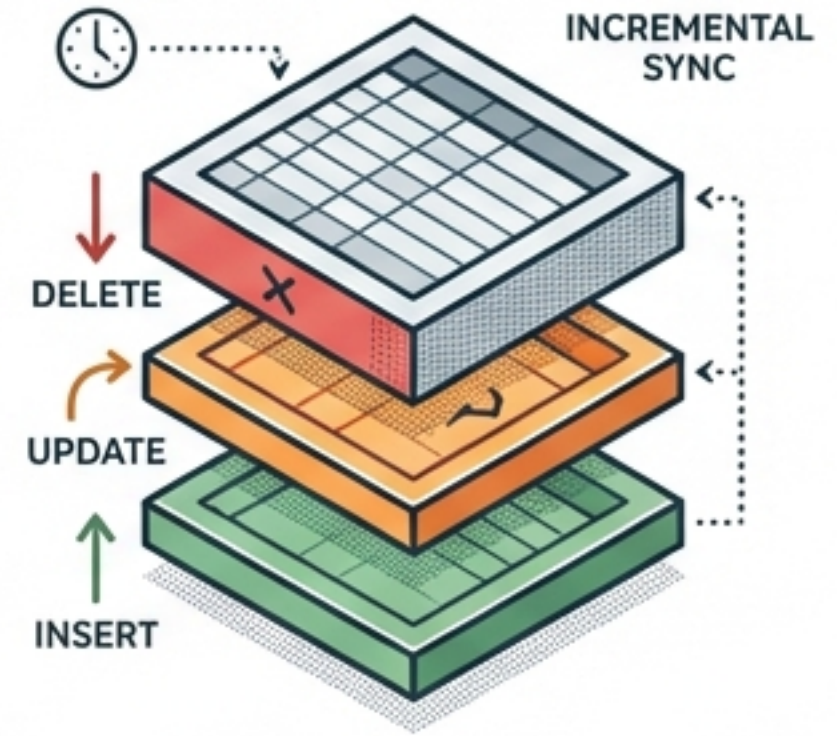
The Problem

Full table refreshes are computationally expensive, slow, and unscalable. Reloading unchanged data wastes compute resources and increases latency.



The Solution

A PySpark CDC pipeline that processes *only* incremental changes.



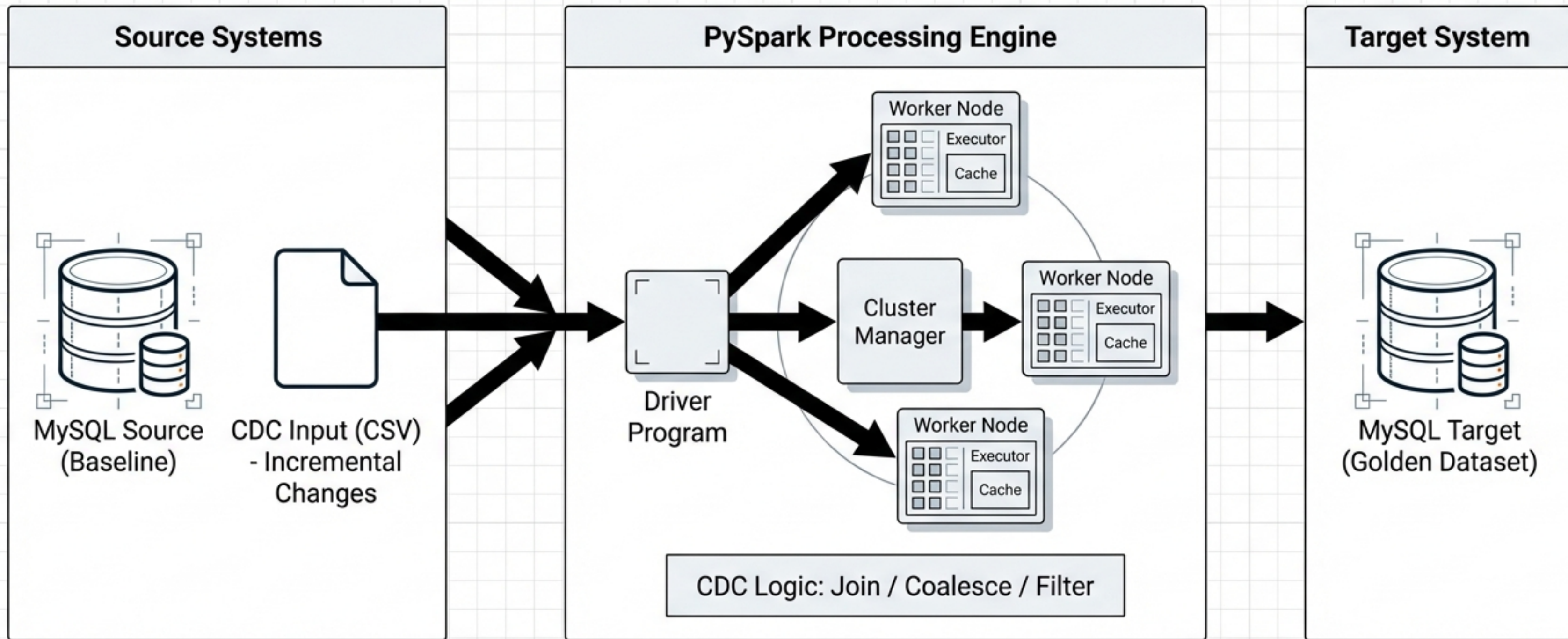
Key Objectives:

Efficiency: Avoid full table reloads.

Precision: Handle Insert, Update, and Delete operations independently.

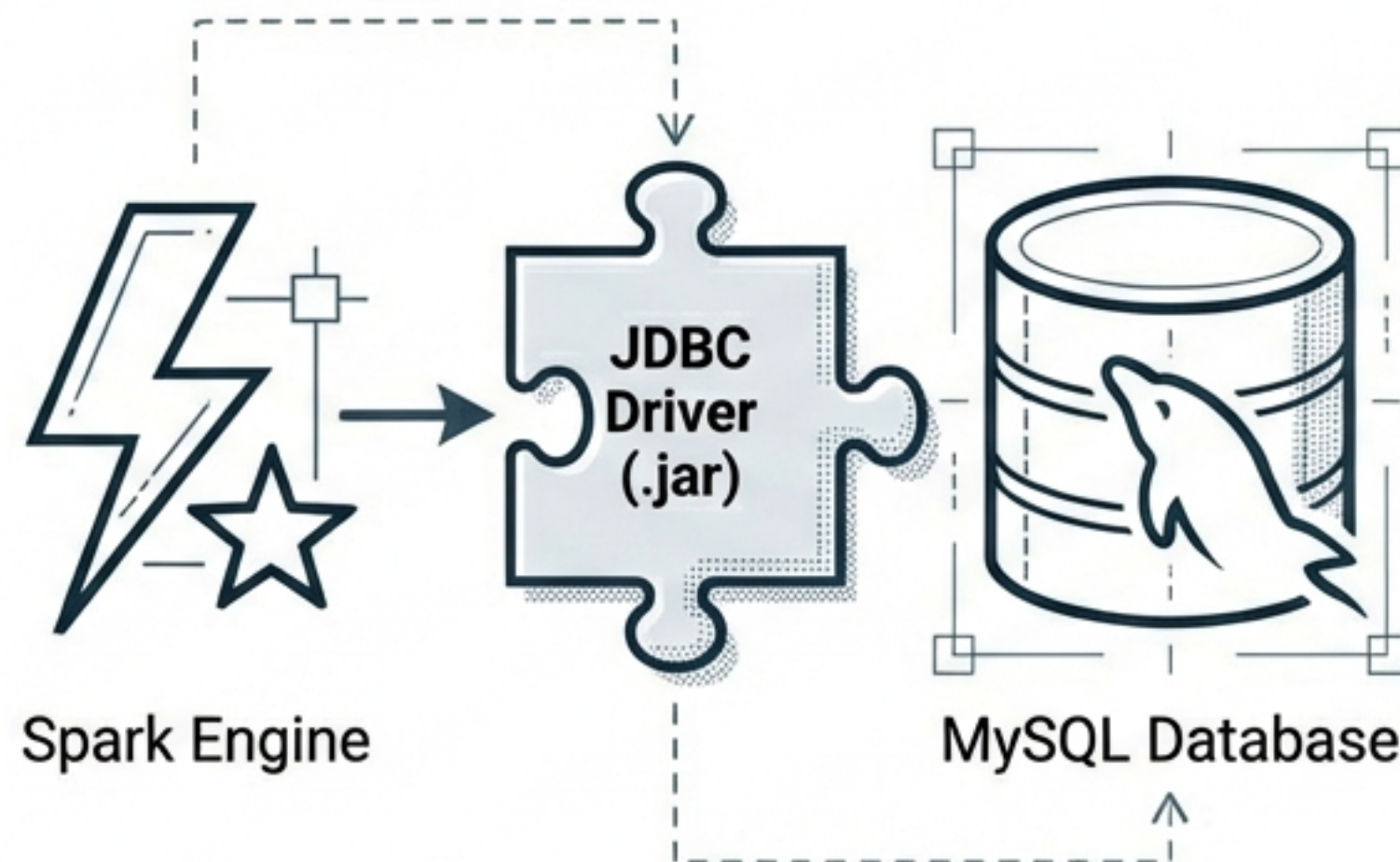
Reliability: Ensure the target table always reflects the latest, accurate state.

High-Level Pipeline Architecture



Ignition: Initializing the Spark Session

```
1 spark = SparkSession.builder \  
2     .appName('CDC_Employee_Project') \  
3     .config(  
4         'spark.jars',  
5         '.../mysql-connector-j-8.4.0.jar'  
6     ) \  
7     .getOrCreate()
```



Without the **JDBC driver JAR**, the Spark engine cannot communicate with the database.

Extraction: Capturing the Source Snapshot

```
1 jdbc_url = 'jdbc:mysql://127.0.0.1:3306/co
2 employee_df = spark.read.jdbc(
3     url=jdbc_url,
4     table='employee_data',
5     properties=properties
6 )
```

MySQL: company_db



JDBC Read

employee_df (Spark DataFrame)

id	name	city	salary

Input: Reads from local MySQL.

Output: Creates an immutable baseline snapshot.

Ingestion: Reading the Change Feed

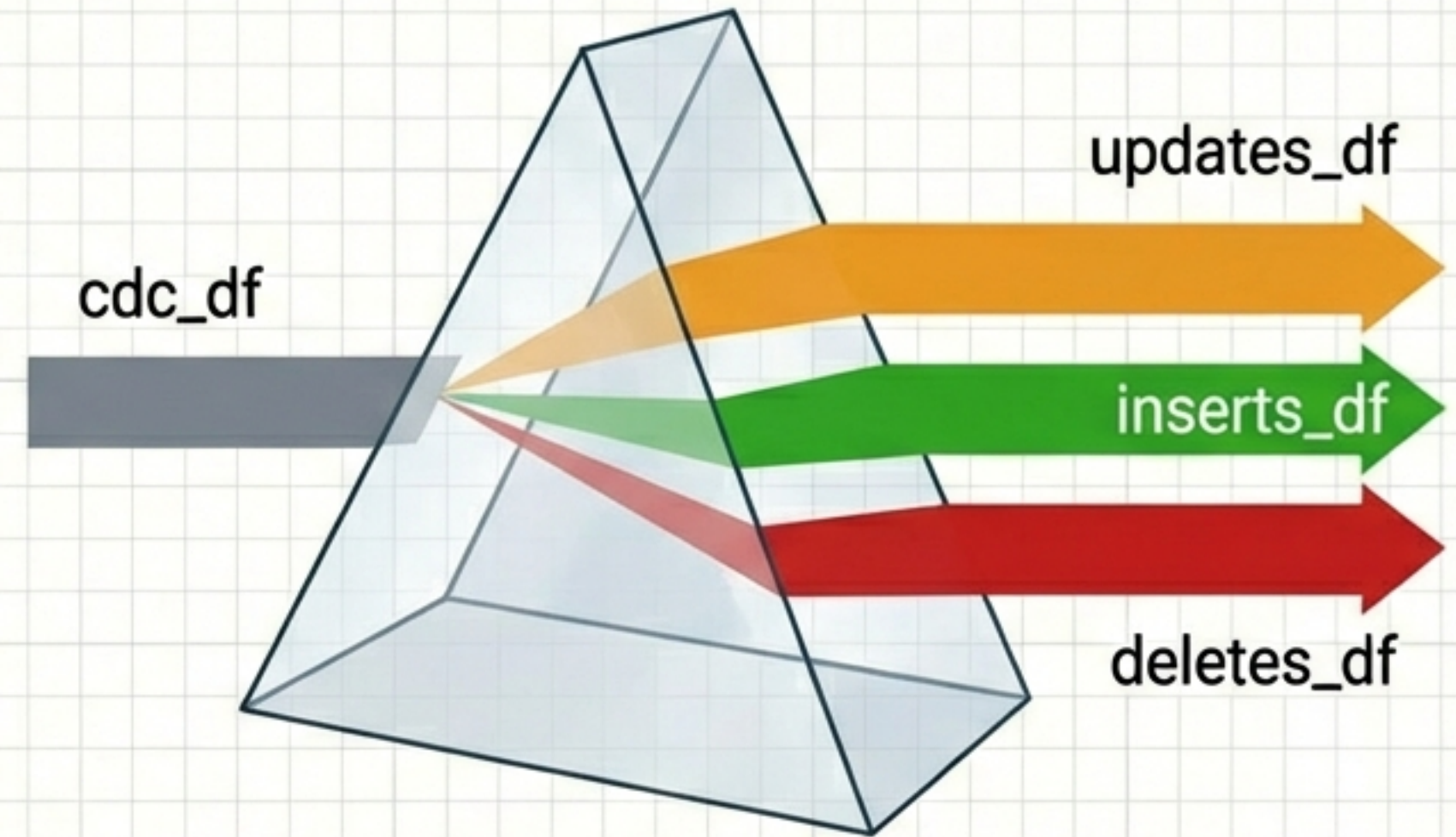
```
1 cdc_df = spark.read \  
2   .option('header', 'true') \  
3   .csv('employees_cdc.csv')
```

emp_id	emp_name	city	salary	operation
105	John Doe	NY	50000	INSERT
102	Jane Smith	LA	60000	UPDATE
109	Bob Lee	NULL	NULL	DELETE

The '**Delta**':
Instructions for
the pipeline.

Segregation: Dividing Operations for Precision

```
1 updates_df = cdc_df.filter(col('operation'
2
3 inserts_df = cdc_df.filter(col('operation'
4 inserts_df = cdc_df.filter(col('operation'
5 deletes_df = cdc_df.filter(col('operation'
6
```



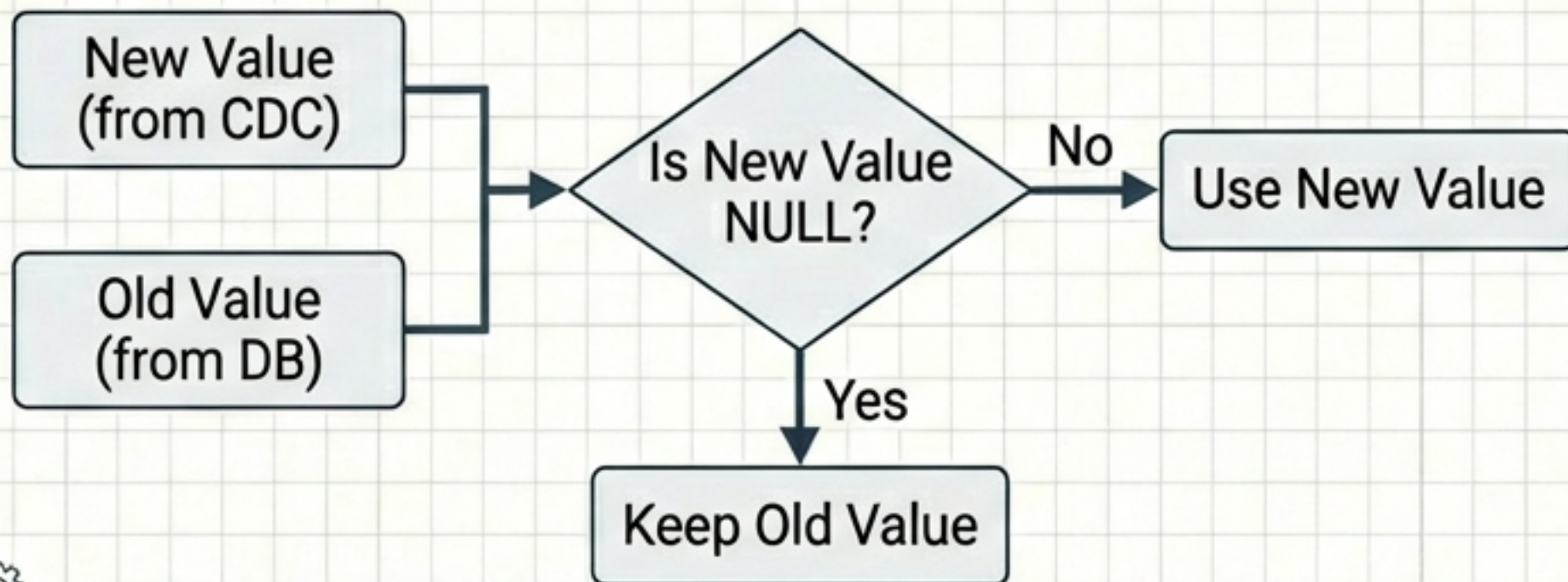
Strategy: Isolate operations to apply specific logic to each type.

The Update Logic: Precision Merging

This slide explains the **`coalesce`** function.

```
1 .join(updates_df, 'emp_id', 'left')
2   coalesce(col('cdc.city'), col('emp.city')).alias('city')
3
```

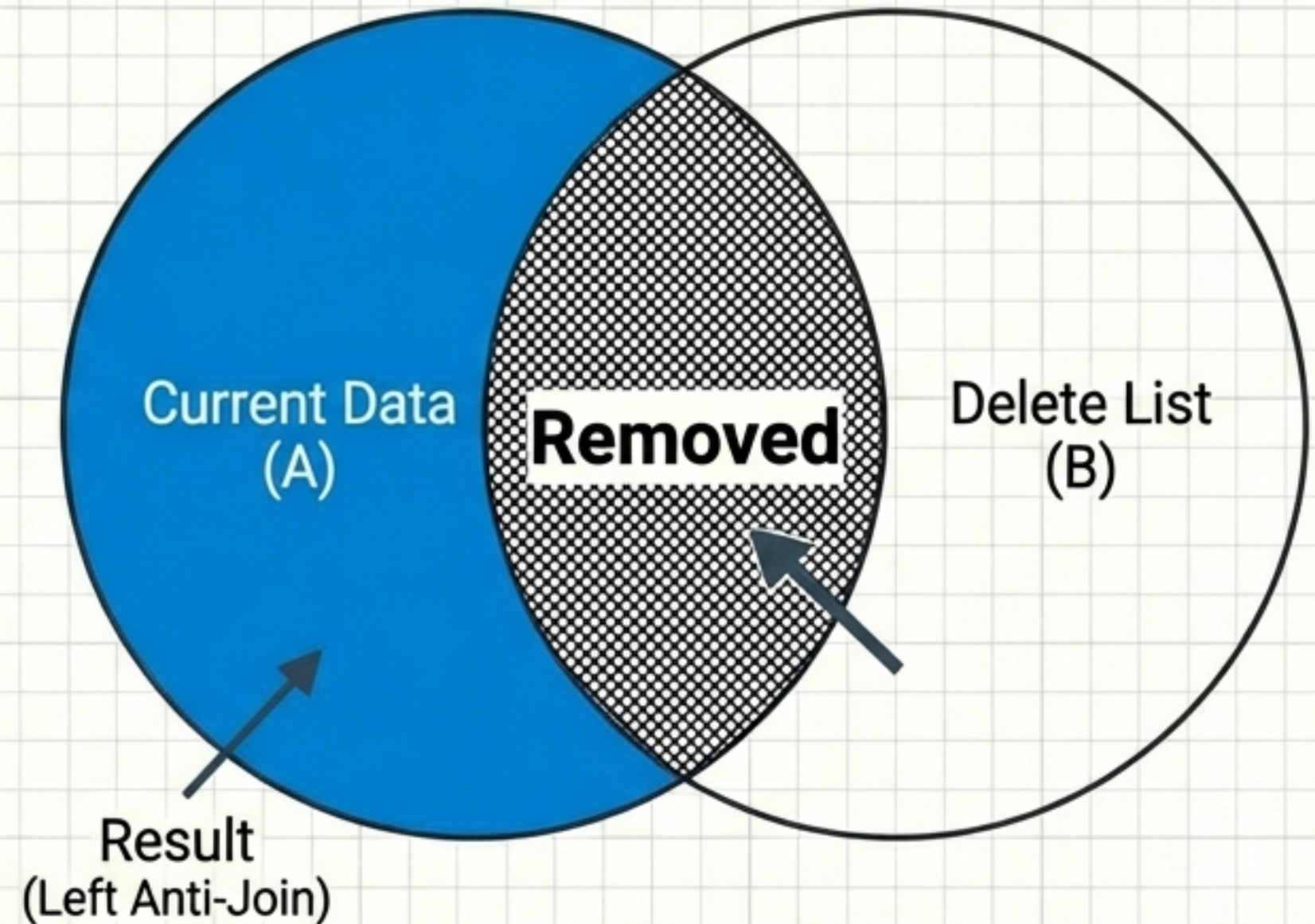
The Coalesce Strategy: `coalesce(new, old)`



	Old	New	Result
1	Paris	London	London
2	Paris	NULL	Paris

The Delete Logic: Exclusion by Key

```
1 after_delete_df = updated_employee_df.join(  
2     deletes_df.select('emp_id'),  
3     on='emp_id',  
4     how='left_anti'  
5 )
```



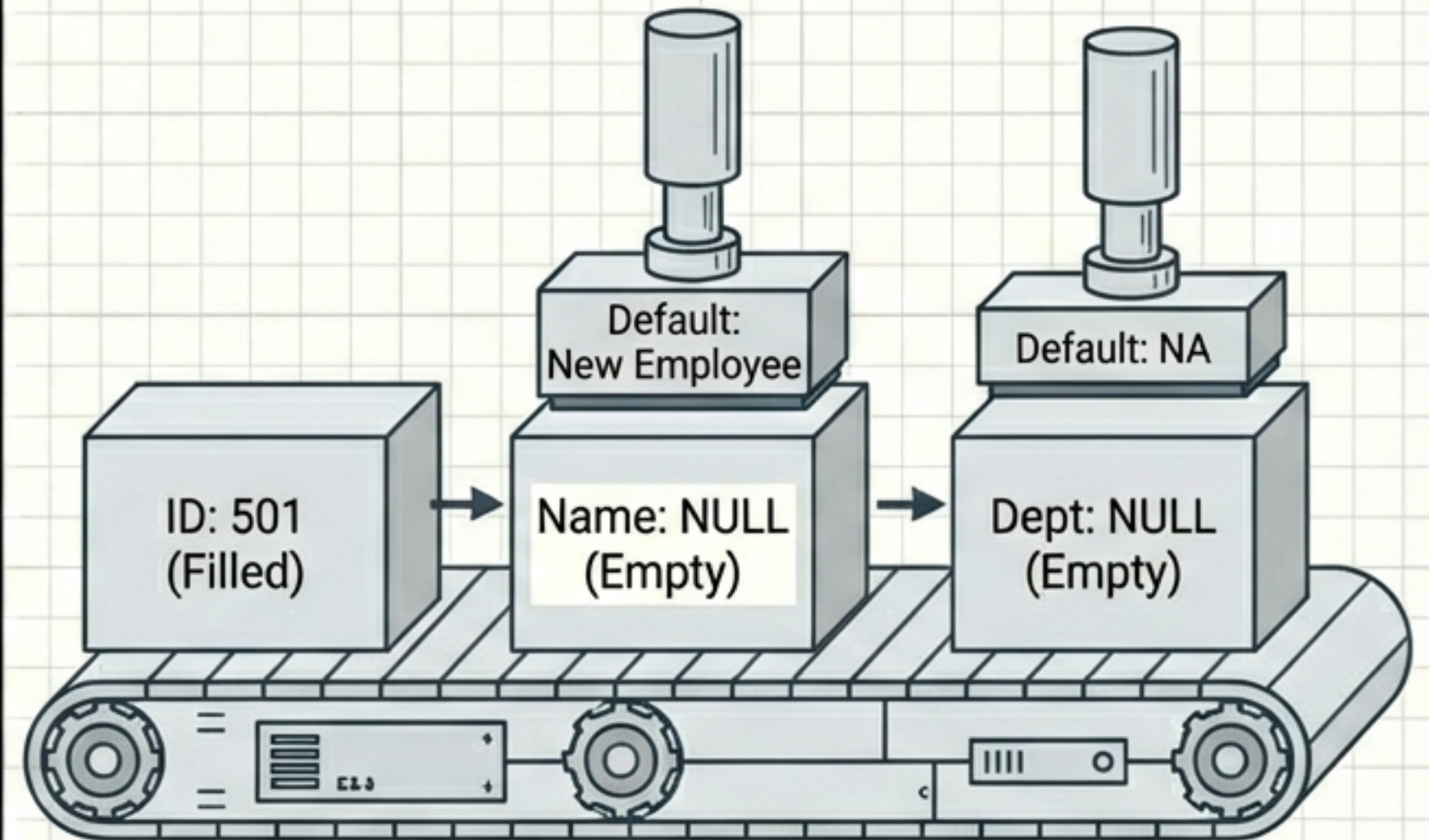
Keep rows from Current Data that do NOT exist in the Delete List.

The Insert Logic: Schema Standardization

Ensuring partial CDC records don't break the target schema.

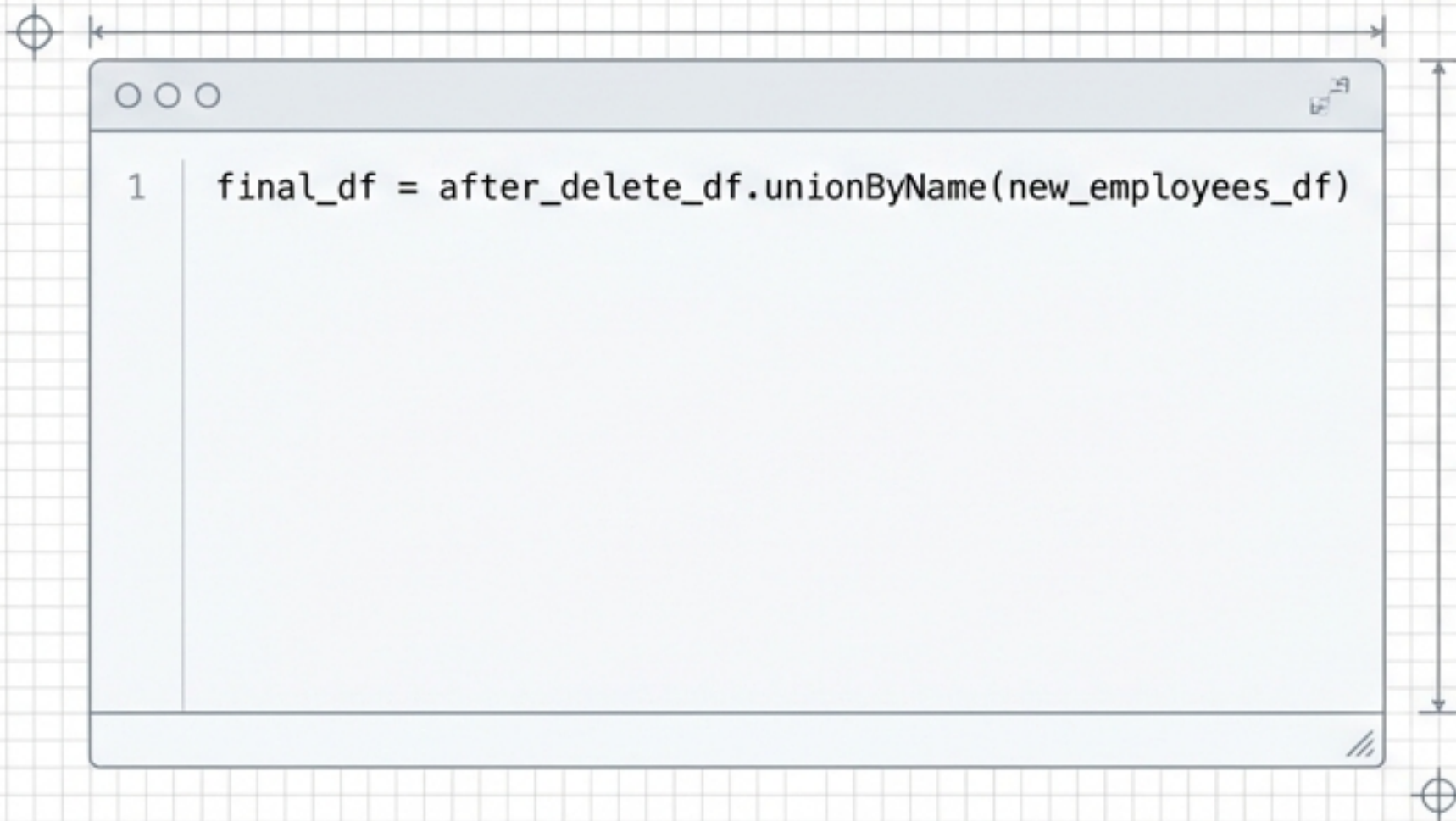
```
1 new_employees_df = inserts_df.select(  
2     col('emp_id'),  
3     lit('New Employee').alias('emp_name'),  
4     lit('NA').alias('dept'),  
5     ...  
6 )
```

Form Stamping

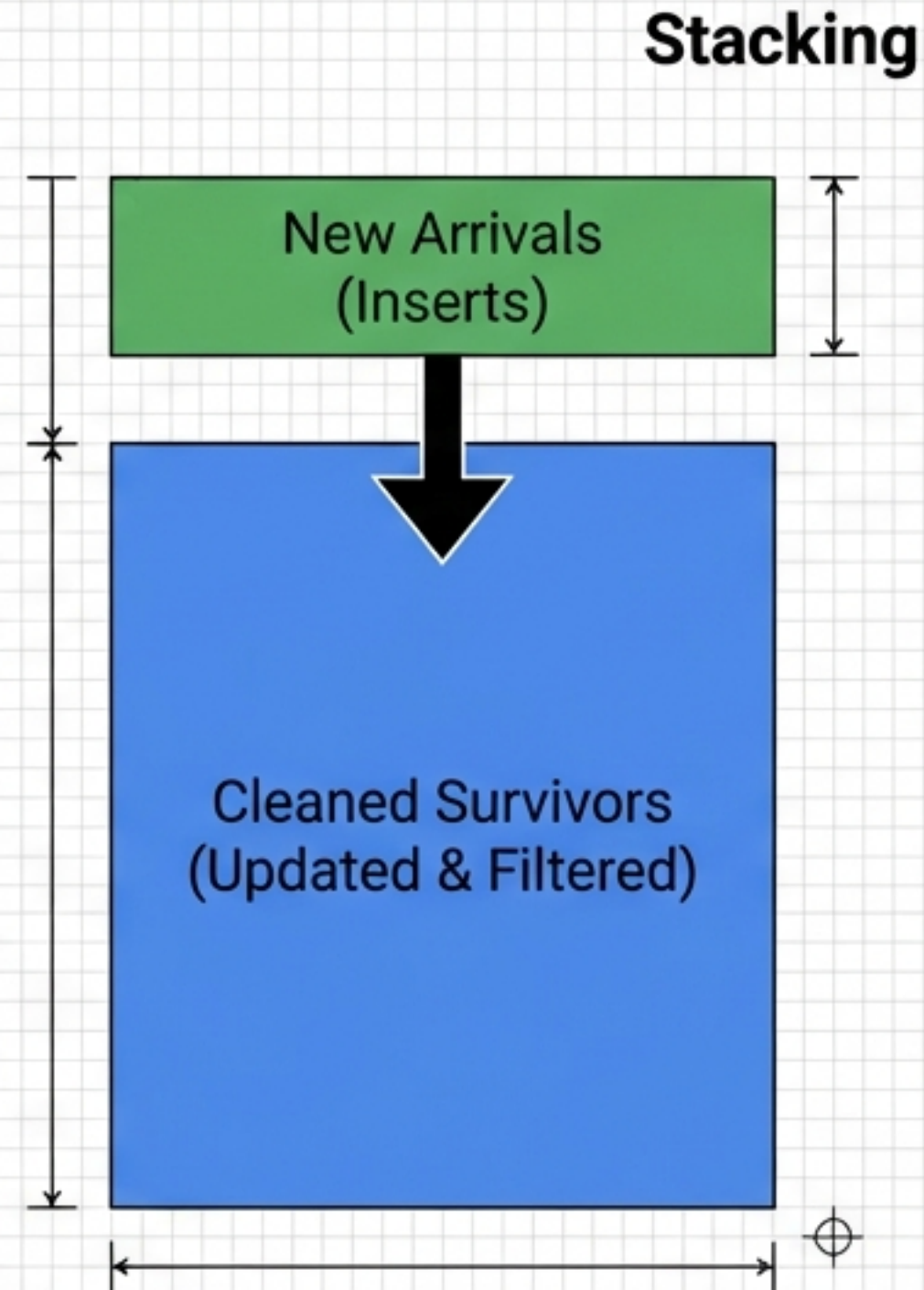


Ensuring partial CDC records don't break the target schema.

Assembly: Constructing the Golden Dataset



```
1 final_df = after_delete_df.unionByName(new_employees_df)
```



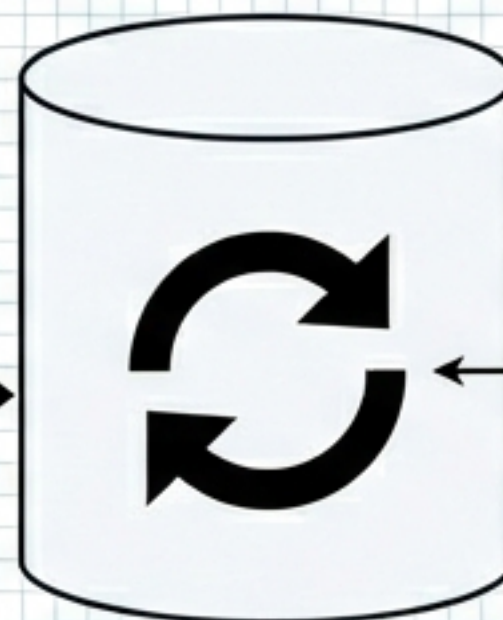
1. Update existing.
2. Delete unwanted.
3. Insert new.
4. Union all.

The Load: Writing to Target

```
1 final_df.write.jdbc(  
2     url=jdbc_url,  
3     table='employee_target',  
4     mode='overwrite',  
5     properties=properties  
6 )
```

Mode: Overwrite

The target table is wiped and replaced with the new, fully calculated Golden Dataset.



MySQL Target

Why This Architecture Wins



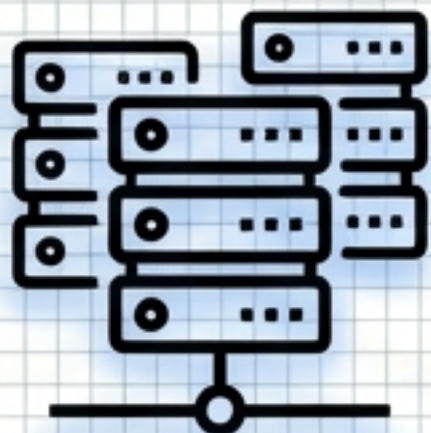
Efficiency

Eliminates processing of unchanged rows, saving compute resources.



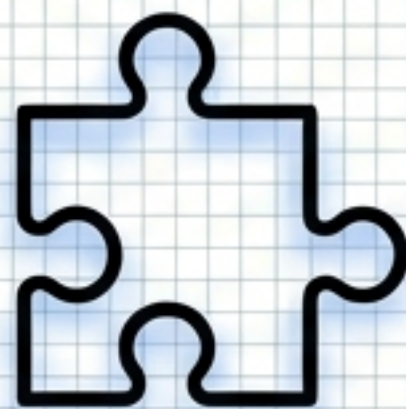
Data Integrity

Handles partial updates safely via coalesce and ensures precise deletions.



Scalability

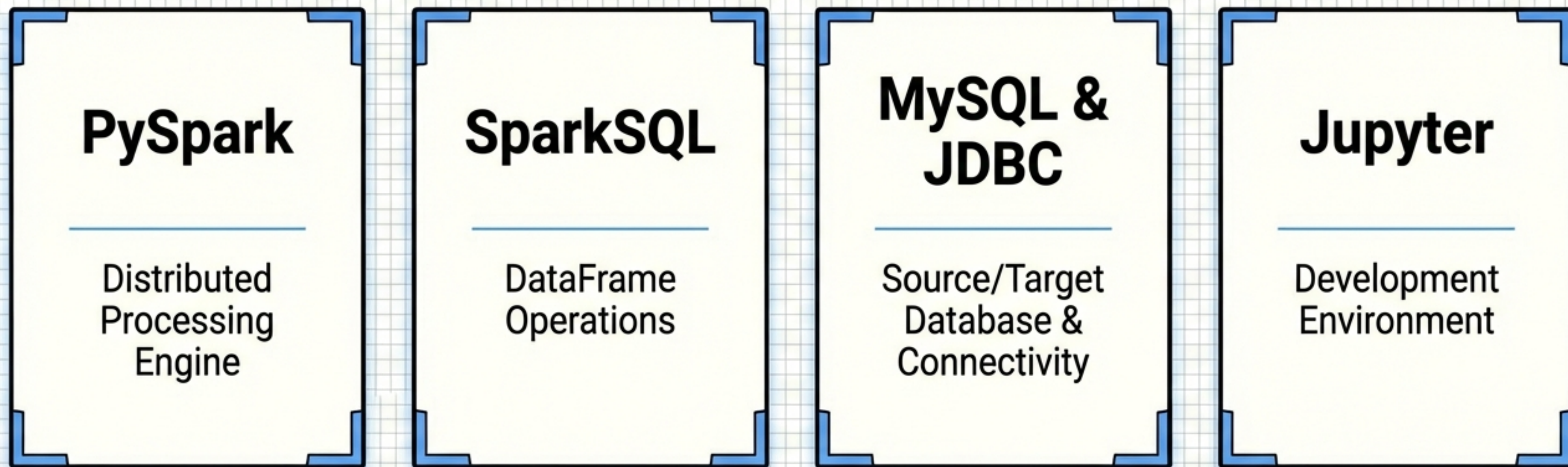
Leverages Spark's distributed processing for Big Data workloads.



Extensibility

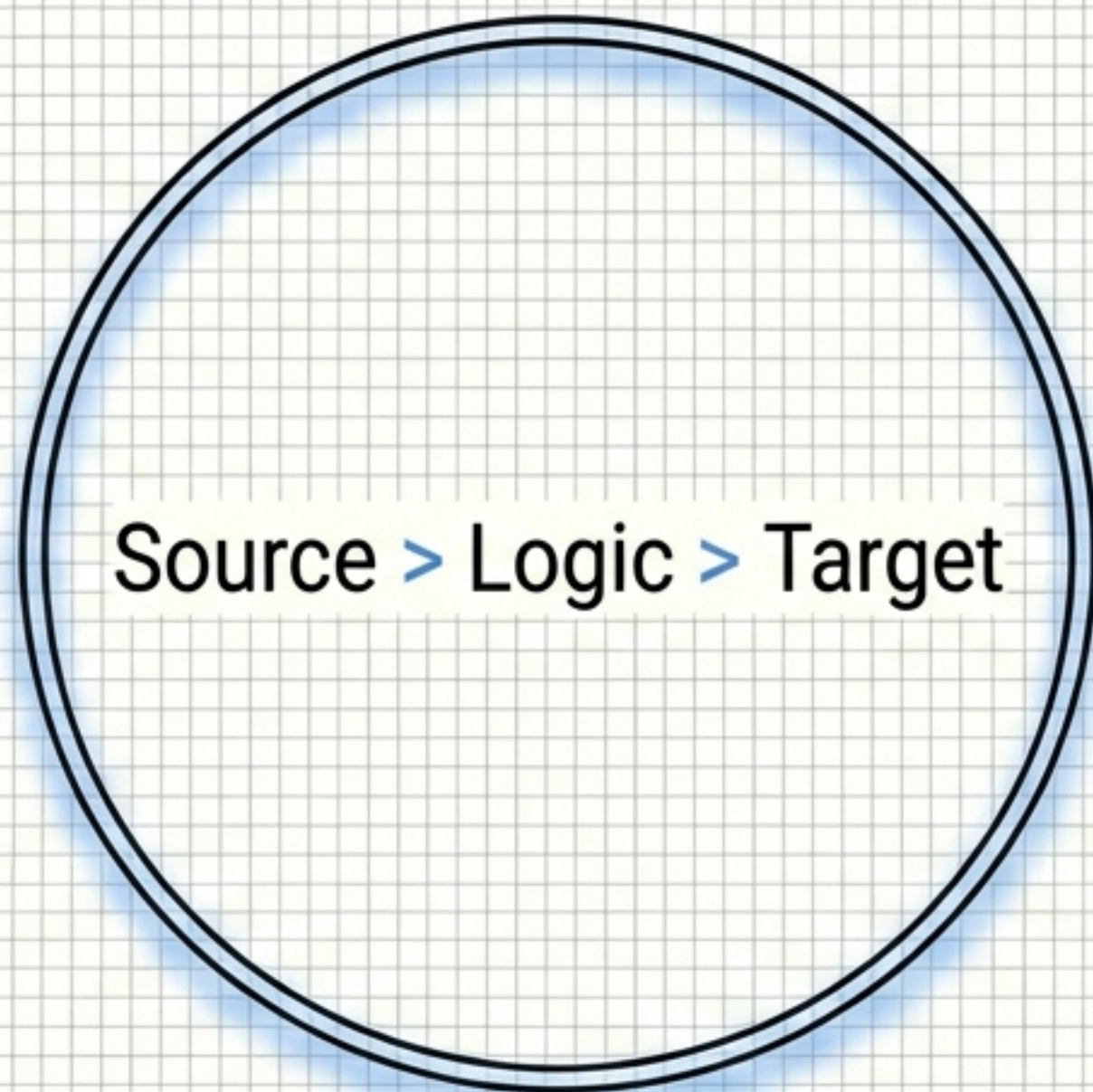
Design mirrors enterprise tools like Kafka/Debezium.

Technology Stack Overview



 Input Format: CSV Change Feed

Intelligent Data Synchronization



Thank You.